



GOVERNMENT TOOL ROOM AND TRAINING CENTRE

(A Scientific & Industrial Research Organization Recognized by Government of India)

A PROJECT REPORT ON “IMAGE PROCESSING TOOLS PROJECT”

Submitted in partial fulfilment of the curriculum requirements prescribed for the

**THIRD SEMESTER OF IMAGE PROCESSING AND
NATURAL LANGUAGE PROCESSING
DIPLOMA IN ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING**

BY

MR.ATHARVSHETTY(8080110)

MS.NAGAVENIPATIL(8080123)

MR.ADINATHVASULKAR(8080102)

MR.ADITYAGODASE(8080103)

**UNDER THE GUIDANCE OF
MR. AMBAPRASAD SIR**

**GOVERNMENT TOOL ROOM AND TRAINING CENTRE
Industrial Estate khanapur road udaymbag,belagavi-590008**



ACKNOWLEDGEMENT

I would like to express my sincere gratitude to **GTTC Belagavi** for providing me with the opportunity and resources to complete this project, **“ImagePro Studio – Image Processing Tools.”**

I am thankful to my **faculty and project mentor** for their valuable guidance, continuous support, and encouragement throughout the development of this project.

I would also like to thank the **Department of AI & Machine Learning** for providing the necessary technical knowledge and laboratory facilities required to successfully complete the project.

Finally, I thank my **parents, friends, and classmates** for their support and motivation during the development of this project.

Thank You!!!!!!

INTRODUCTION

ImagePro Studio is a web-based **Image Processing Tools** application developed using **Python, Flask, OpenCV, NumPy, HTML, CSS, and JavaScript**. The main purpose of this project is to provide a simple and interactive platform for performing fundamental digital image processing operations.

The application allows users to **upload an image from their computer or capture an image using a camera**. After selecting an image, users can apply different image processing techniques such as **grayscale conversion, binary conversion, RGB to HSV conversion, histogram equalization, brightness and contrast adjustment, average filtering, Gaussian filtering, Laplacian sharpening, Sobel edge detection, and Canny edge detection**.

The Flask backend handles the image-processing requests, while **OpenCV and NumPy** perform the required image operations. The processed image is then displayed to the user, who can preview and download the result in supported formats such as **JPG, PNG, and BMP**.

The project is mainly developed for **educational and academic purposes**. It helps students understand the practical implementation of fundamental image processing concepts and demonstrates how **Python, OpenCV, and web technologies** can be combined to create a useful image-processing application.

OBJECTIVES OF THE PROJECT

- To understand the basic concepts of digital image processing.
- To implement image processing operations using OpenCV.
- To develop a user-friendly web-based image processing application.
- To allow image input through upload and camera capture.
- To perform image enhancement, filtering, color conversion, and edge detection.
- To display and download processed images.
- To demonstrate the practical use of Python, Flask, OpenCV, and NumPy.

TECHNOLOGY USED

The **ImagePro Studio** project uses the following technologies:

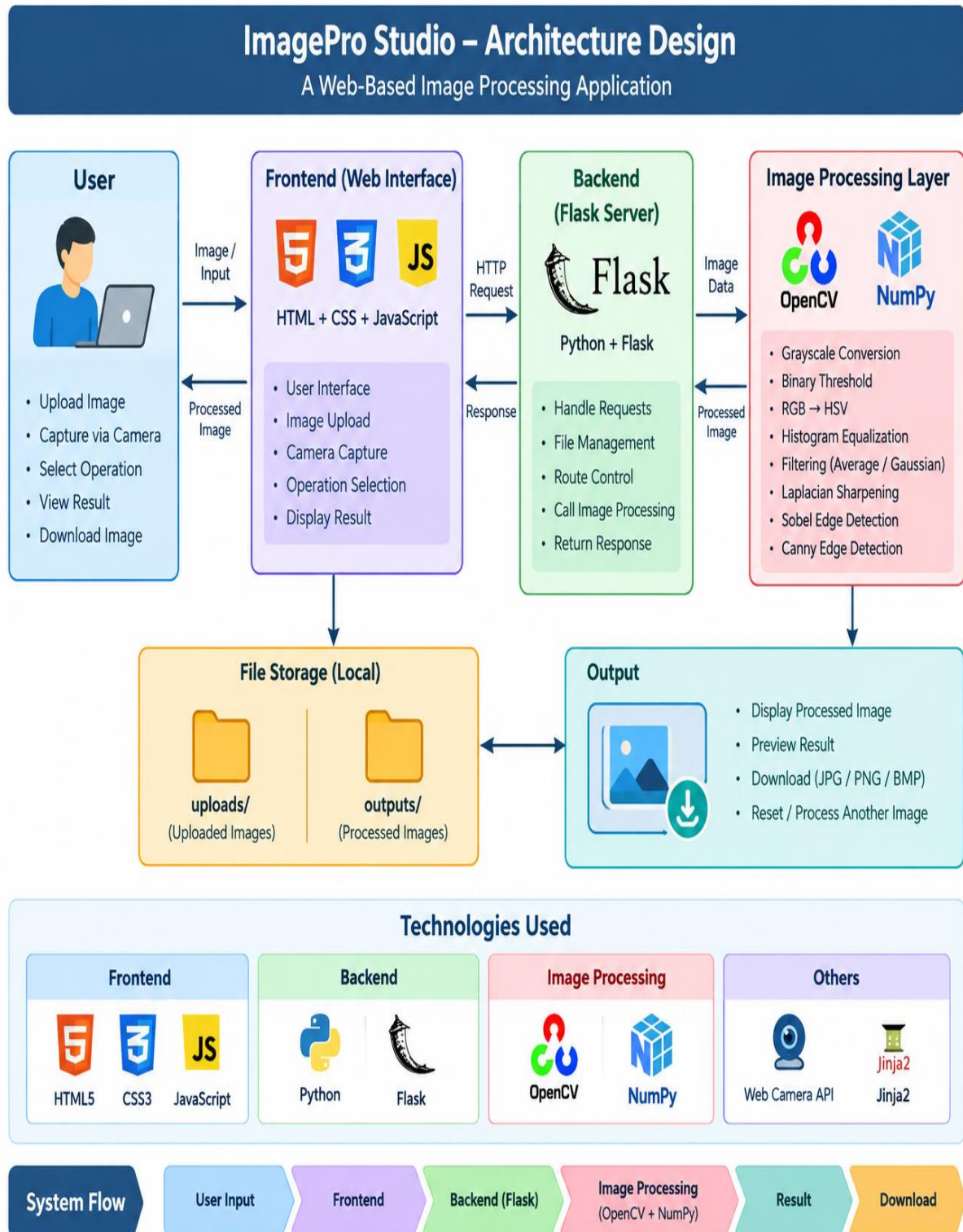
Technology	Purpose
Python	Main programming language
Flask	Develops the web application and handles backend requests
OpenCV	Performs image processing operations
NumPy	Handles image data and numerical operations
HTML5	Creates the structure of the web pages
CSS3	Provides styling and layout
JavaScript	Provides interactive features and camera functionality
Web Camera API	Captures images directly from the browser camera
Jinja2	Connects Flask backend data with HTML templates

Main Technologies

- **Frontend:** HTML5, CSS3, JavaScript
- **Backend:** Python, Flask
- **Image Processing:** OpenCV
- **Numerical Processing:** NumPy
- **Input:** Image Upload and Web Camera
- **Output:** JPG, PNG, and BMP images

ARCHITECTURE DESIGN

The **ImagePro Studio** follows a simple **three-layer web application architecture** consisting of the frontend, backend, and image-processing layer.



WORKING OF THE ARCHITECTURE

1. **User Input:** The user uploads an image or captures an image using the camera.
2. **Frontend:** HTML, CSS and JavaScript provide the user interface and send the image to the Flask server.
3. **Backend:** Flask receives the image and manages the processing request.
4. **Image Processing:** OpenCV and NumPy perform the selected image-processing operation.
5. **Output:** The processed image is returned to the frontend and displayed to the user.
6. **Download:** The user can download the processed image in **JPG, PNG, or BMP** format.

FILE STRUCTURE

The **ImagePro Studio** project is organized into the following files and folders:

ImageProStudio_fixed/

```
|
|
|— app.py
|— requirements.txt
|— README.md
|
|— templates/
|   |— index.html
|
|— static/
|   |— style.css
|   |— script.js
|
|— uploads/
|   |— Uploaded images
|
|— outputs/
|   |— Processed images
```

IMAGE PROCESSING OPERATIONS

1. Grayscale Conversion

Method: Converts a color image into a grayscale image.

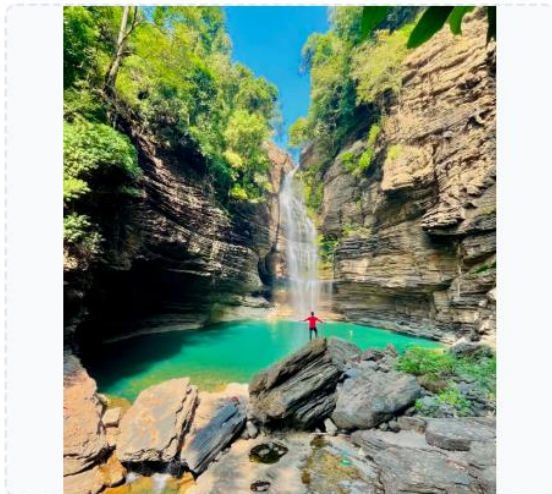
Syntax:

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

Output Produced: Grayscale image.

Original & Processed Image

Original Image



Processed Result



Processing Operations

Choose an operation

2. Convert to Grayscale



Process Image

Reset

2. Binary Image

Method: Converts a grayscale image into black and white using thresholding.

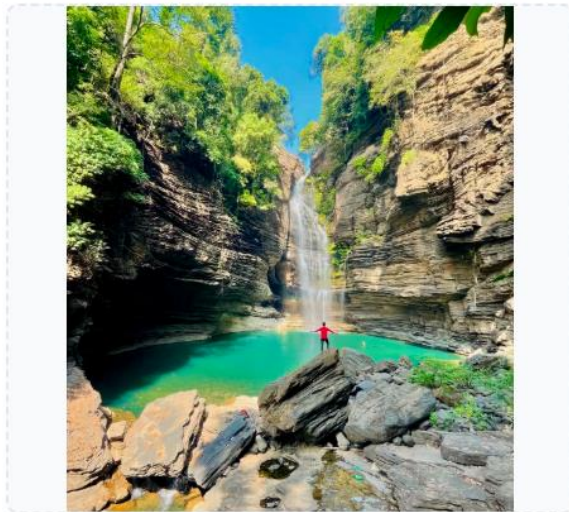
Syntax:

```
_, binary = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)
```

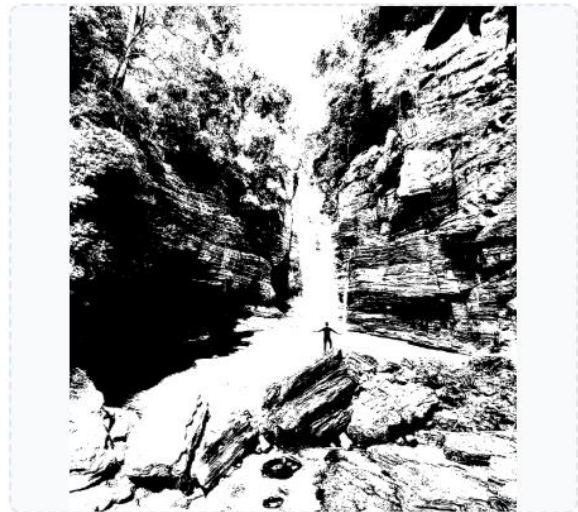
Output Produced: Binary black-and-white image.

Original & Processed Image

Original Image



Processed Result



Processing Operations

Choose an operation

3. Convert to Binary



Threshold: 127



Process Image

Reset

3. RGB to HSV

Method: Converts the image from BGR/RGB color space to HSV color space.

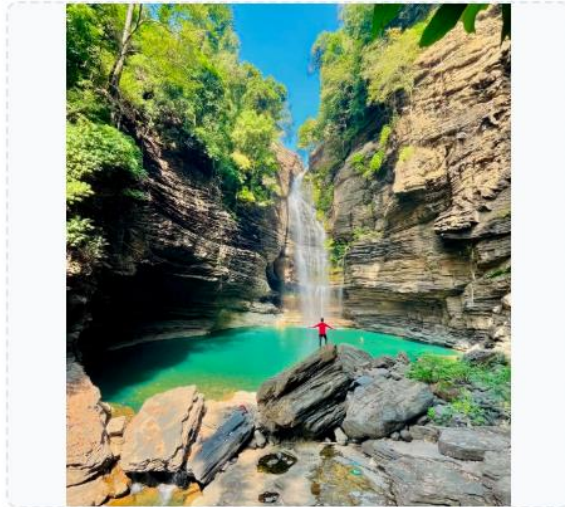
Syntax:

```
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
```

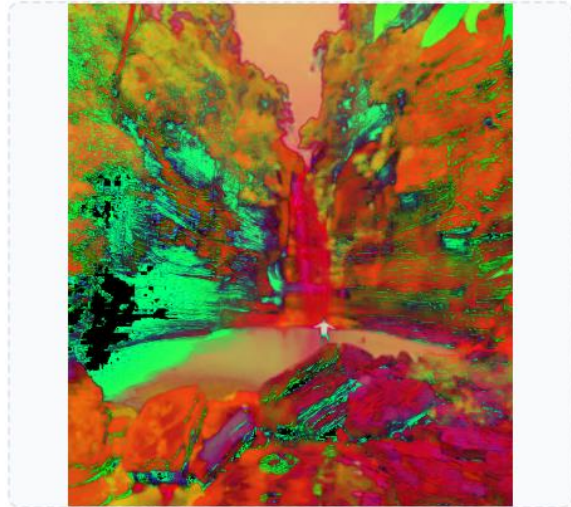
Output Produced: HSV color image.

Original & Processed Image

Original Image



Processed Result



Processing Operations

Choose an operation

6. RGB to HSV

Process Image

Reset

4. Histogram Equalization

Method: Improves the contrast of a grayscale image.

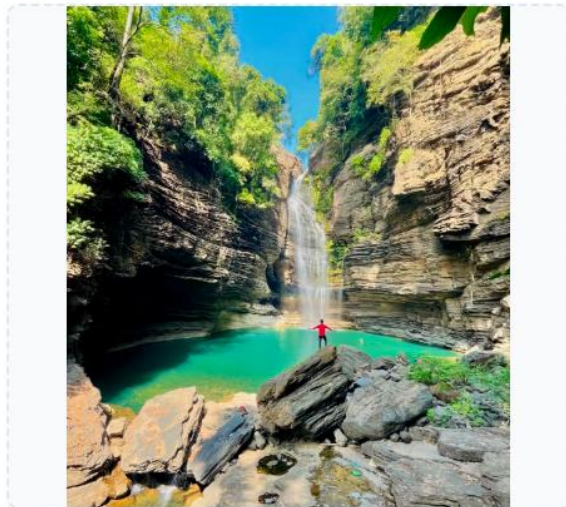
Syntax:

```
equalized = cv2.equalizeHist(gray)
```

Output Produced: Contrast-enhanced image.

Original & Processed Image

Original Image



Processed Result



Processing Operations

Choose an operation

7. Histogram Equalization

Process Image

Reset

5. Brightness and Contrast Adjustment

Method: Changes the brightness and contrast of an image.

Syntax:

```
result = cv2.convertScaleAbs(img, alpha=1.2, beta=30)
```

Output Produced: Image with adjusted brightness and contrast.

Original & Processed Image

Original Image



Processed Result



Processing Operations

Choose an operation

8. Brightness & Contrast

Brightness: 78



Contrast: 1.5



Process Image

Reset

6. Average Filter

Method: Smooths the image by averaging neighboring pixels.

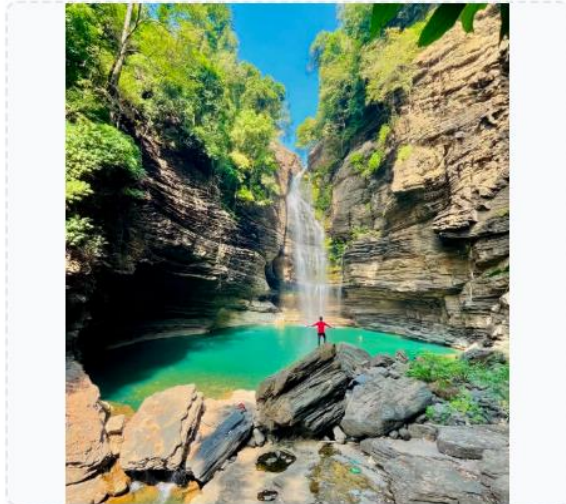
Syntax:

```
result = cv2.blur(img, (5, 5))
```

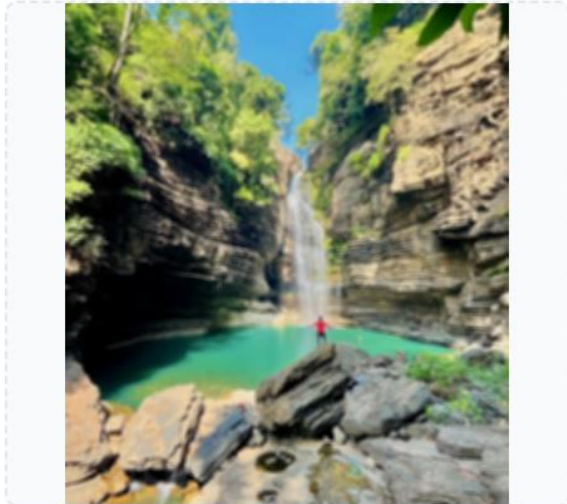
Output Produced: Smoothed/noise-reduced image.

Original & Processed Image

Original Image



Processed Result



Processing Operations

Choose an operation

9. Average Filter

Kernel Size: 13



Process Image

Reset

7. Gaussian Filter

Method: Smooths the image using a Gaussian kernel.

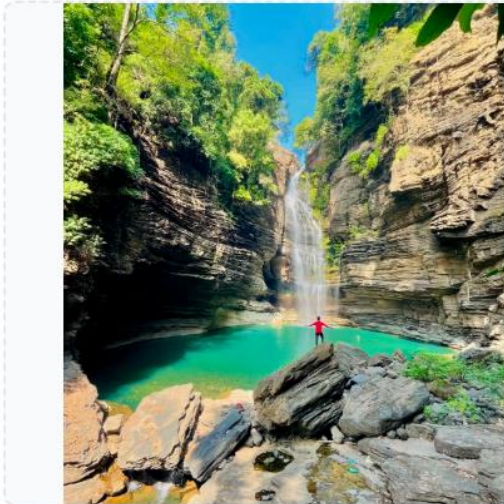
Syntax:

```
result = cv2.GaussianBlur(img, (5, 5), 0)
```

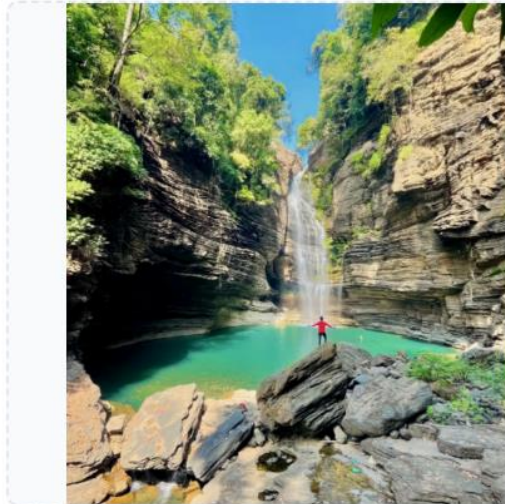
Output Produced: Smooth and noise-reduced image.

Original & Processed Image

Original Image



Processed Result



Processing Operations

Choose an operation

10. Gaussian Filter

Kernel Size: 5



Process Image

Reset

8. Laplacian Sharpening

Method: Enhances edges and fine details in an image.

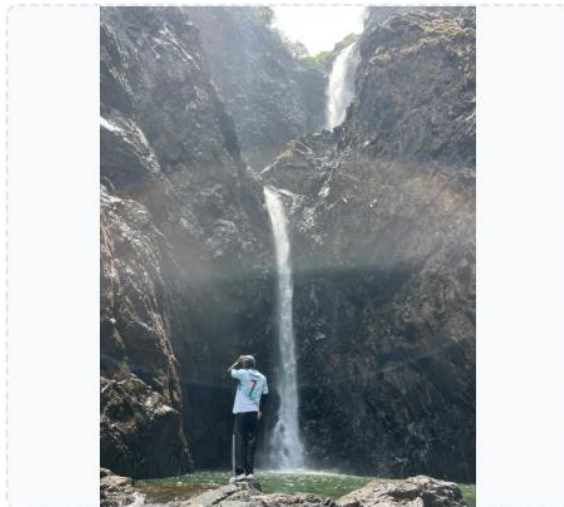
Syntax:

```
laplacian = cv2.Laplacian(img, cv2.CV_64F)
```

Output Produced: Edge-enhanced/sharpened image.

Original & Processed Image

Original Image



Processed Result



Processing Operations

Choose an operation

11. Laplacian Sharpening

Process Image

Reset

9. Sobel Edge Detection

Method: Detects edges in horizontal or vertical directions.

Syntax:

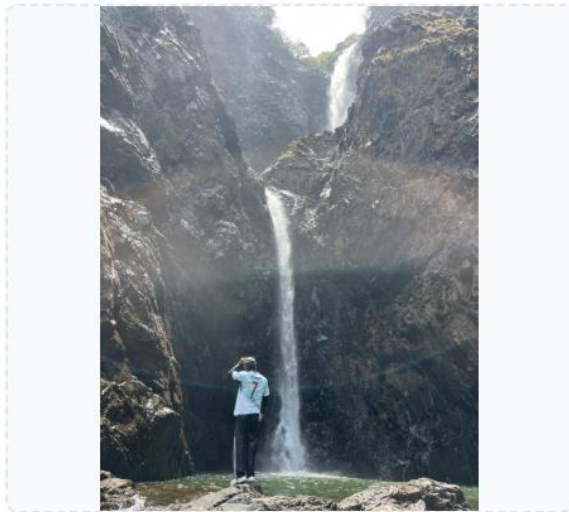
```
sobelx = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
```

```
sobely = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
```

Output Produced: Image showing detected edges.

Original & Processed Image

Original Image



Processed Result



Processing Operations

Choose an operation

12. Sobel Edge Detection

Direction

Both (X & Y)

Process Image

Reset

10. Canny Edge Detection

Method: Detects strong and important edges in an image.

Syntax:

```
edges = cv2.Canny(gray, 100, 200)
```

Output Produced: Binary edge-detected image.

Original & Processed Image

Original Image

Processed Result



Processing Operations

Choose an operation

13. Canny Edge Detection ▾

Lower Threshold: 100

Upper Threshold: 200

Process Image

Reset

CONCLUSION

The **ImagePro Studio – Image Processing Tools** project successfully provides a simple and user-friendly web-based platform for performing basic image processing operations. The application combines **Python, Flask, OpenCV, NumPy, HTML, CSS, and JavaScript** to process images efficiently.

The system allows users to **upload images or capture images using a camera**, apply various processing techniques such as **grayscale, binary thresholding, color conversion, histogram equalization, filtering, sharpening, and edge detection**, and view the processed results.

This project helped in understanding the **practical implementation of digital image processing concepts** and demonstrated how image-processing techniques can be integrated into a web application. Overall, **ImagePro Studio** is a useful educational project for learning and demonstrating fundamental image processing operations using OpenCV.

----ATHARV SHETTY

FINAL YEAR DAIML 5TH SEM